Individual Sprint Reports - Sprint 4

Sprint: 4
Dates: 07 Apr - 21 Apr
Sprint Goal: Implement a backend database for the Smart Study Tracker to store tasks persistently and support future API requests.

User Story Assigned:

- As a student, I want my tasks to be stored in a database, so that they persist across sessions and devices.
- As a student, I want the system to support task creation, editing, deletion, and retrieval through a backend API, so that the front-end remains dynamic and consistent.

Tasks Assigned:

Task ID	Task Description	Owner	Estimated Time (hrs)	Actual Time (hrs)	Acceptance Criteria	Evidence link
T10	Set up Flask app with SQLite database to store tasks	Maxwell Emmanuel	3	4	Database stores tasks with title, module, deadline and completion status; persists across refresh	Smart Study Trac…
T11	Implement API endpoints for task CRUD operations	Maxwell Emmanuel	3	5	Endpoints allow front end to fetch, add, edit, delete tasks; tested through Postman/Hoppscotch	Smart Study Trac…
T12	Connect front end to API using fetch()	Maxwell Emmanuel	3	3.5	Task list updates dynamically from database; all previous front end features function with	Smart Study Trac…

					API	

Acceptance Criteria & Testing

Database Integration (US10/US11)

Test ID	Given	When	Then	Result
Test 1	A new task is submitted from front end	User submits task via form	Task is saved to the database and appears in task list	Pass
Test 2	Task exists in database	User edits task via front end	Changes and reflected in database and UI	Pass
Test 3	Task exists in database	User deletes task via front end	Task is removed from database and UI	Pass
Task 4	User refreshes page	Task data is retrieved from database	All tasks display correctly	Pass

Reflection

This sprint focused on implementing a backend database and API integration for the Smart Study Tracker. I set up a Flask server with SQLite to persist task data and created endpoints for creating, editing, deleting and fetching tasks. Integrating the front end with the API required updating the JavaScript logic to fetch tasks dynamically instead of using localStorage.

The only challenge I faced was understanding how different HTTP methods (GET, POST, PUT, DELETE) work, as some routes returned “Method Not Allowed” when accessed incorrectly through the browser. I resolved this by testing endpoints using the correct request methods and by adding additional routes for easier testing. I also made sure that the database correctly stored and updated task data without duplication or loss.

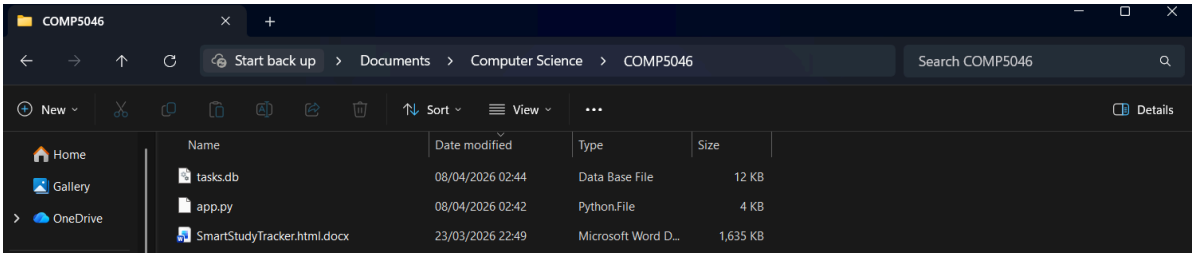
Overall, this sprint improved my understanding of backend development and APIs. The system now supports data storage and provides a structured way for the front end to interact with the database, making a solid foundation for full system integration.

The implementation of the backend database and API improved the Smart Study Tracker by enabling persistent storage and data management. Through the use of Flask and SQLite, the system now supports full CRUD operations, allowing tasks to be created, retrieved, updated and deleted.

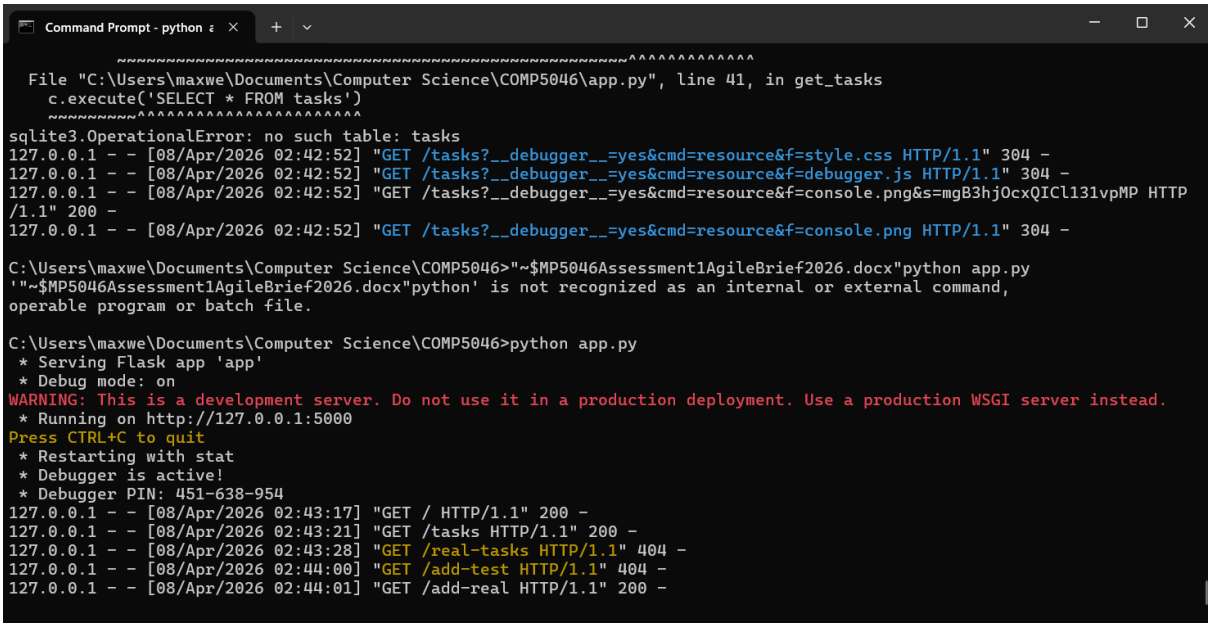
Testing the API endpoints confirmed that the system behaves reliably and maintains data consistency across operations. This backend structure provides a maintainable solution, making sure that feature features such as full front end integration can be built on.

Evidence

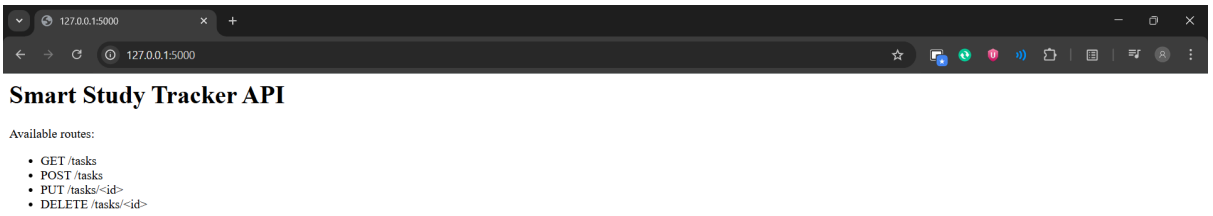
Database initialisation (tasks.db in project folder):



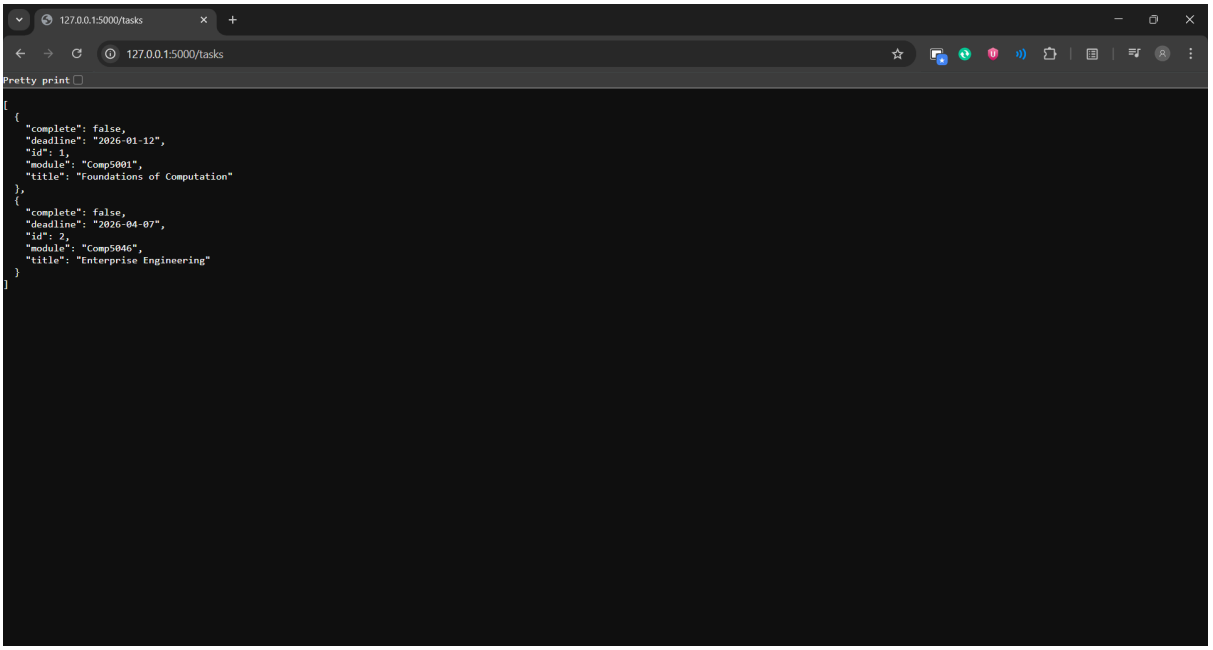
Flask server running:


Command Prompt - python a
~~~
~~~

API Homepage:



GET /tasks Endpoint:



/add-real Endpoint:

Showing a success message in the browser, tasks can be added to the database without using an external client.